Boolean Truth Table Exercise — Selecting a Generative Hardware Design Toolchain

Purpose. Decide which of three curriculum-derived programs, or which combined toolchain, best serves as a manual/semi-automated generative program for **Hardware Design Engineering** using **Python + KiCad**, three-dimensional block design, superposition, dimension matching, and collision theory.

> This is an exercise, not a certified manufacturing workflow. The demo is deliberately educational: it generates abstract block envelopes, interface points, fit checks, and optional KiCad placement scaffolding. Real manufacture requires datasheets, tolerances, materials, insulation ratings, thermal analysis, electrical safety review, and DFM/DFA sign-off.

---

1. The Three Candidate Programs

Symbol	Program	Interpreted role in this exercise	Main design pattern extracted
A	Program 1: baseline safety-critical permutation generator	Clean, auditable generator core	`Configuration → OutputSink → safe bounded generation → FileSink`
B	Program 2: enhanced compact fixed-width generator	Minimal parametric enumerator	Fixed-width decimal/token generation with bounded output
C	Program 3: advanced `configurator_v2` fabric	User-specifiable generative fabric	Config/CLI/UI-defined objects, flows, separators, prefix/suffix, targets

---

2. Boolean Feature Predicates

Evaluate each program or program-combination against these predicates.

Predicate	Meaning	Hardware-design interpretation
G	General generative breadth	Can generate many alternatives, not just one fixed object
Cnf	Configurability	User can specify objects, dimensions, flows, outputs
S	Safety/verification posture	Uses bounded arithmetic, validation, preconditions, fail-fast behaviour
I	Interface abstraction	Has a separable sink/output or tool boundary
K	KiCad/Python bridgeability	Can be mapped naturally into Python objects and KiCad footprints/placement
D	Dimension matching	Can represent width/height/depth and connector/interface matching
X	Collision checking	Can test 3D/2D overlap before assembly

Predicate	Meaning	Hardware-design interpretation
M	Manufacture/assembly metadata	Can preserve naming, prefix/suffix, target, process notes, or export artifacts

A candidate is considered **hardware-toolchain-ready** when:

```
READY = G ∧ Cnf ∧ S ∧ I ∧ K ∧ D ∧ X ∧ M
```

A candidate is considered **teaching-ready but not production-ready** when:

```
TEACHING = (S ∧ I) ∨ (G ∧ D ∧ X)
```

---

3. Boolean Truth Table

Use `1 = present/satisfied`, `0 = absent/not sufficiently satisfied`.

Row	A	B	C	Candidate toolchain	G	Cnf	S	I	K	D	X	M	READY	TEACHING	Decision
1	1	0	0	**A only**	1	0	1	1	1	0	0	0	0	1	Good safety kernel, weak hardware configurability
2	0	1	0	**B only**	1	0	1	1	1	0	0	0	0	1	Minimal generator, useful for numeric series and IDs
3	0	0	1	**C only**	1	1	1	1	1	1	0	1	0	1	Best single program, but needs explicit collision engine
4	1	1	0	**A+B**	1	0	1	1	1	0	0	0	0	1	Strong bounded enumerator pair; not enough object metadata
5	1	0	1	**A+C**	1	1	1	1	1	1	0	1	0	1	Strongest two-program base; add

Row	A	B	C	Candidate toolchain	G	Cnf	S	I	K	D	X	M	READY	TEACHING	Decision
															collision module
6	0	1	1	**B+C**	1	1	1	1	1	1	0	1	0	1	Good compact + configurable fabric; add safety audit from A
7	1	1	1	**A+B+C hybrid**	1	1	1	1	1	1	1	1	1	1	**Best choice: hybrid hardware design generator**

Answer Key

The best answer is:

```
A ∧ B ∧ C = READY
```

The **hybrid toolchain** is best because it combines:

1. **A:** auditable safety-critical generator structure, overflow guards, and output abstraction.
2. **B:** compact deterministic fixed-width enumeration, useful for component codes, pin labels, footprints, and part variants.
3. **C:** configurable object/flow fabric, useful for user-named hardware objects, config files, CLI/UI invocation, and flexible outputs.
4. **Python + KiCad layer:** adds the missing hardware-specific geometry, interface, 3D envelope, collision, and board-placement semantics.

---

4. Hardware Design Pattern Philosophy

The exercise treats each electrical component as a **design pattern instance**.

```
Component Pattern =
    identity
  + functional role
  + 3D envelope
  + interface points
  + material/process metadata
  + electrical constraints
  + assembly constraints
  + collision/dimension checks
  + KiCad footprint/placement output
```

Component set to build

```
wire
cable
connector
battery cell
supercapacitor
switch
relay
resistor
capacitor
inductor
transformer
fuse
circuit breaker
```

Four engineering lenses

Lens	Question	Example
Superposition	What happens when candidate layouts are overlaid in the same coordinate frame?	Overlay connector, fuse, relay, and battery keep-out zones.
Dimension matching	Do interfaces physically and electrically match?	Cable diameter matches connector bore; pin pitch matches footprint pads.
Collision theory	Do occupied volumes intersect?	Relay case must not intersect transformer volume or battery keep-out.
DFM/DFA	Can it be manufactured and assembled?	Sufficient clearance, labeled orientation, process notes, access space.

---

5. Interactive Demo — Python Geometry + Optional KiCad Export

What the demo does

The demo builds a tiny abstract hardware assembly:

- Models each required component as a 3D axis-aligned block.
- Adds interface points such as terminals, pins, or cable endpoints.
- Checks **dimension matching** between paired interfaces.
- Checks **collision** with AABB overlap.
- Produces a simple placement manifest.
- Optionally creates a KiCad PCB scaffold if run inside an environment where `pcbnew` is available.

Install/use notes

- Geometry-only demo runs with standard Python 3.
- KiCad export section is optional and version-sensitive.
- For modern KiCad plugin work, prefer KiCad's IPC API / `kicad-python` where available.
- Legacy `pcbnew` scripting examples are included as a fallback pattern for learning.

---

6. Demo Code

Save as:

```
hardware_block_demo.py
```

```
from __future__ import annotations

from dataclasses import dataclass, field
from itertools import combinations
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Tuple
import json
import math

Vec3 = Tuple[float, float, float]


@dataclass(frozen=True)
class Interface:
    """A physical/electrical connection point on a component block."""
    name: str
    kind: str                 # e.g. wire_end, pin, lug, terminal, pad
    position: Vec3            # local component coordinates in mm
    nominal_diameter: float   # mm
    voltage_class: str = "ELV"
    current_class: str = "signal"


@dataclass
class BlockComponent:
    """3D block approximation of a hardware component."""
    name: str
    category: str
    origin: Vec3              # world-space lower-left-bottom corner in mm
    size: Vec3                # width, depth, height in mm
    interfaces: List[Interface] = field(default_factory=list)
    material: str = "unspecified"
    process: str = "manual assembly"
    notes: str = ""

    @property
    def max_corner(self) -> Vec3:
        return tuple(self.origin[i] + self.size[i] for i in range(3))  # type: ignore

    def world_interface(self, iface: Interface) -> Vec3:
        return tuple(self.origin[i] + iface.position[i] for i in range(3))  # type:
ignore


def aabb_collides(a: BlockComponent, b: BlockComponent, clearance: float = 0.0) ->
bool:
    """Collision theory: axis-aligned bounding-box overlap with optional clearance."""
```

```
    amin, amax = a.origin, a.max_corner
    bmin, bmax = b.origin, b.max_corner

    return all(
        (amin[i] - clearance) < bmax[i] and
        (amax[i] + clearance) > bmin[i]
        for i in range(3)
    )


def distance(p: Vec3, q: Vec3) -> float:
    return math.sqrt(sum((p[i] - q[i]) ** 2 for i in range(3)))


def dimension_match(a: Interface, b: Interface, tolerance: float = 0.25) -> bool:
    """Dimension matching: simple round-interface compatibility."""
    return abs(a.nominal_diameter - b.nominal_diameter) <= tolerance


def make_component_library() -> Dict[str, BlockComponent]:
    """Abstract 3D block library for all requested components."""
    return {
        "wire": BlockComponent(
            "wire", "conductor", (0, 0, 0), (30, 1, 1),
            [Interface("end_A", "wire_end", (0, 0.5, 0.5), 1.0),
             Interface("end_B", "wire_end", (30, 0.5, 0.5), 1.0)],
            material="copper + insulation",
            process="cut, strip, crimp/solder"
        ),
        "cable": BlockComponent(
            "cable", "multi_conductor", (0, 4, 0), (40, 3, 3),
            [Interface("end_A", "cable_end", (0, 1.5, 1.5), 3.0),
             Interface("end_B", "cable_end", (40, 1.5, 1.5), 3.0)],
            material="copper bundle + jacket",
            process="cut, strip, strain-relief"
        ),
        "connector": BlockComponent(
            "connector", "interconnect", (42, 4, 0), (8, 6, 5),
            [Interface("bore", "socket", (0, 3, 2.5), 3.1),
             Interface("pin_1", "pad", (7, 2, 0), 1.0),
             Interface("pin_2", "pad", (7, 4, 0), 1.0)],
            material="polymer housing + plated contacts",
            process="insert, solder, inspect"
        ),
        "battery_cell": BlockComponent(
            "battery_cell", "energy_storage", (0, 14, 0), (18, 65, 18),
            [Interface("positive", "terminal", (18, 32.5, 9), 5.0, "battery"),
             Interface("negative", "terminal", (0, 32.5, 9), 5.0, "battery")],
            material="cell chemistry dependent",
            process="holder or welded tabs; observe polarity"
        ),
        "supercapacitor": BlockComponent(
            "supercapacitor", "energy_storage", (24, 14, 0), (10, 20, 18),
            [Interface("positive", "lead", (5, 3, 0), 0.8),
             Interface("negative", "lead", (5, 17, 0), 0.8)],
            material="aluminium can",
```

```
            process="through-hole insert; polarity check"
        ),
        "switch": BlockComponent(
            "switch", "control", (55, 0, 0), (12, 6, 5),
            [Interface("common", "pad", (2, 3, 0), 1.0),
             Interface("throw", "pad", (10, 3, 0), 1.0)],
            material="plastic actuator + metal contacts",
            process="panel/PCB mount"
        ),
        "relay": BlockComponent(
            "relay", "electromechanical_control", (72, 0, 0), (20, 15, 12),
            [Interface("coil_A", "pad", (3, 3, 0), 1.0),
             Interface("coil_B", "pad", (3, 12, 0), 1.0),
             Interface("contact_COM", "pad", (17, 3, 0), 1.2),
             Interface("contact_NO", "pad", (17, 12, 0), 1.2)],
            material="sealed relay package",
            process="PCB insert; creepage/clearance review"
        ),
        "resistor": BlockComponent(
            "resistor", "passive", (42, 16, 0), (8, 3, 3),
            [Interface("lead_A", "lead", (0, 1.5, 1.5), 0.6),
             Interface("lead_B", "lead", (8, 1.5, 1.5), 0.6)],
            material="film or wirewound",
            process="place, solder, trim"
        ),
        "capacitor": BlockComponent(
            "capacitor", "passive", (42, 24, 0), (8, 5, 8),
            [Interface("lead_A", "lead", (3, 2.5, 0), 0.6),
             Interface("lead_B", "lead", (5, 2.5, 0), 0.6)],
            material="ceramic/electrolytic/film",
            process="place; polarity if required"
        ),
        "inductor": BlockComponent(
            "inductor", "magnetic", (55, 24, 0), (12, 10, 8),
            [Interface("lead_A", "lead", (0, 5, 0), 0.8),
             Interface("lead_B", "lead", (12, 5, 0), 0.8)],
            material="winding + core",
            process="place; magnetic clearance"
        ),
        "transformer": BlockComponent(
            "transformer", "magnetic", (72, 24, 0), (24, 20, 18),
            [Interface("primary_A", "lug", (2, 3, 0), 1.2),
             Interface("primary_B", "lug", (2, 17, 0), 1.2),
             Interface("secondary_A", "lug", (22, 3, 0), 1.2),
             Interface("secondary_B", "lug", (22, 17, 0), 1.2)],
            material="copper winding + laminated/ferrite core",
            process="mount; isolation and thermal review"
        ),
        "fuse": BlockComponent(
            "fuse", "protection", (42, 38, 0), (18, 5, 5),
            [Interface("clip_A", "clip", (0, 2.5, 2.5), 2.0),
             Interface("clip_B", "clip", (18, 2.5, 2.5), 2.0)],
            material="fuse body + clips",
            process="clip holder; replaceability access"
        ),
        "circuit_breaker": BlockComponent(
```

```
            "circuit_breaker", "protection", (64, 38, 0), (28, 18, 20),
            [Interface("line", "terminal", (0, 6, 10), 3.5),
             Interface("load", "terminal", (28, 6, 10), 3.5)],
            material="breaker housing + contacts",
            process="panel mount; torque terminals"
        ),
    }


def superpose_variants(base: Iterable[BlockComponent], dx: float = 0.0, dy: float =
0.0) -> List[BlockComponent]:
    """
    Superposition: overlay or offset candidate assemblies.
    dx=dy=0 means true overlay; nonzero offsets create candidate alternatives.
    """
    out = []
    for item in base:
        moved = BlockComponent(
            name=f"{item.name}_variant",
            category=item.category,
            origin=(item.origin[0] + dx, item.origin[1] + dy, item.origin[2]),
            size=item.size,
            interfaces=item.interfaces,
            material=item.material,
            process=item.process,
            notes=f"superposed offset dx={dx}, dy={dy}"
        )
        out.append(moved)
    return out


def report_collisions(parts: List[BlockComponent], clearance: float = 0.5) ->
List[Tuple[str, str]]:
    hits = []
    for a, b in combinations(parts, 2):
        if aabb_collides(a, b, clearance=clearance):
            hits.append((a.name, b.name))
    return hits


def report_interface_matches(parts: Dict[str, BlockComponent]) -> List[Dict[str,
object]]:
    """Small set of intentional interface fit checks."""
    checks = [
        ("cable", "end_B", "connector", "bore"),
        ("wire", "end_B", "switch", "common"),
        ("fuse", "clip_B", "circuit_breaker", "line"),
        ("supercapacitor", "positive", "capacitor", "lead_A"),
    ]

    rows = []
    for comp_a, iface_a, comp_b, iface_b in checks:
        a = parts[comp_a]
        b = parts[comp_b]
        ia = next(i for i in a.interfaces if i.name == iface_a)
        ib = next(i for i in b.interfaces if i.name == iface_b)
```

```
            rows.append({
                "a": f"{comp_a}.{iface_a}",
                "b": f"{comp_b}.{iface_b}",
                "diameter_a_mm": ia.nominal_diameter,
                "diameter_b_mm": ib.nominal_diameter,
                "dimension_match": dimension_match(ia, ib),
                "world_distance_mm": round(distance(a.world_interface(ia),
b.world_interface(ib)), 2),
            })
    return rows


def export_manifest(parts: Dict[str, BlockComponent], path: Path) -> None:
    data = []
    for part in parts.values():
        data.append({
            "name": part.name,
            "category": part.category,
            "origin_mm": part.origin,
            "size_mm": part.size,
            "interfaces": [
                {
                    "name": i.name,
                    "kind": i.kind,
                    "local_position_mm": i.position,
                    "nominal_diameter_mm": i.nominal_diameter,
                    "voltage_class": i.voltage_class,
                    "current_class": i.current_class,
                }
                for i in part.interfaces
            ],
            "material": part.material,
            "process": part.process,
            "notes": part.notes,
        })
    path.write_text(json.dumps(data, indent=2), encoding="utf-8")


def optional_kicad_export(parts: Dict[str, BlockComponent], out_path: Path) -> str:
    """
    Optional KiCad scaffold.

    This deliberately uses a conservative legacy pcbnew pattern if available:
    - Create/load a board
    - Add simple text labels as placement proxies
    - Save the board

    In KiCad 9+, modern plugin development should prefer the IPC API / kicad-python
    where possible. This fallback is only a teaching scaffold.
    """
    try:
        import pcbnew  # type: ignore
    except Exception as exc:
        return f"KiCad pcbnew not available; skipped board export: {exc}"

    board = pcbnew.BOARD()
```

```
    for idx, part in enumerate(parts.values()):
        # PCB_TEXT exists in recent pcbnew versions; older versions may differ.
        text = pcbnew.PCB_TEXT(board)
        text.SetText(part.name)
        text.SetPosition(pcbnew.VECTOR2I(int(part.origin[0] * 1_000_000),
int(part.origin[1] * 1_000_000)))
        text.SetLayer(pcbnew.F_SilkS)
        board.Add(text)

    board.Save(str(out_path))
    return f"Saved KiCad scaffold to {out_path}"


def main() -> None:
    parts = make_component_library()

    print("\n=== Hardware Block Manifest ===")
    for p in parts.values():
        print(f"{p.name:18s} origin={p.origin} size={p.size} process={p.process}")

    print("\n=== Dimension Matching Checks ===")
    for row in report_interface_matches(parts):
        print(row)

    print("\n=== Collision Checks with 0.5 mm clearance ===")
    collisions = report_collisions(list(parts.values()), clearance=0.5)
    if not collisions:
        print("No collisions detected.")
    else:
        for a, b in collisions:
            print(f"COLLISION/CLEARANCE FAIL: {a} <-> {b}")

    print("\n=== Superposition Test ===")
    overlay = superpose_variants([parts["relay"], parts["transformer"]], dx=0, dy=0)
    overlay_hits = report_collisions([parts["relay"], parts["transformer"], *overlay],
clearance=0.0)
    for hit in overlay_hits:
        print(f"Superposed overlap: {hit}")

    manifest = Path("hardware_blocks_manifest.json")
    export_manifest(parts, manifest)
    print(f"\nWrote {manifest}")

    print(optional_kicad_export(parts, Path("hardware_blocks.kicad_pcb")))


if __name__ == "__main__":
    main()
```

---

7. Expected Interactive Output Pattern

When you run:

```
python hardware_block_demo.py
```

you should see sections like:

```
=== Hardware Block Manifest ===
wire               origin=(0, 0, 0) size=(30, 1, 1) process=cut, strip, crimp/solder
cable              origin=(0, 4, 0) size=(40, 3, 3) process=cut, strip, strain-relief
...

=== Dimension Matching Checks ===
{'a': 'cable.end_B', 'b': 'connector.bore', 'diameter_a_mm': 3.0, 'diameter_b_mm':
3.1, 'dimension_match': True, ...}
{'a': 'supercapacitor.positive', 'b': 'capacitor.lead_A', 'diameter_a_mm': 0.8,
'diameter_b_mm': 0.6, 'dimension_match': True, ...}

=== Collision Checks with 0.5 mm clearance ===
No collisions detected.
```

If KiCad's Python module is available, it also attempts to write:

```
hardware_blocks.kicad_pcb
```

Otherwise it still writes:

```
hardware_blocks_manifest.json
```

---

8. Design-Manufacture-Assembly Mapping

Component	Design variables	Manufacture variables	Assembly variables	Demo abstraction
Wire	length, gauge, insulation, endpoint type	cut length, strip length, conductor material	bend radius, termination	3D line/block + two wire-end interfaces
Cable	conductor count, jacket diameter, shield	cut, strip, jacket removal	strain relief, connector fit	larger block + cable-end interface
Connector	pitch, pin count, bore/socket size	plated contacts, housing	mating direction, keying	block + bore + pads
Battery cell	chemistry, voltage, capacity, terminals	cell procurement, holder/weld tabs	polarity, spacing, thermal access	energy block + positive/negative terminals

Component	Design variables	Manufacture variables	Assembly variables	Demo abstraction
Supercapacitor	capacitance, ESR, voltage rating	can package, lead forming	polarity, clearance	cylindrical-as-block + leads
Switch	poles/throws, current rating	contact material, actuator	panel or PCB mounting	control block + terminals
Relay	coil voltage, contact rating	sealed package	coil/contact separation	block + coil/contact pads
Resistor	value, tolerance, power	film/wirewound, lead forming	placement, heat spacing	passive block + leads
Capacitor	value, dielectric, polarity	package, forming	polarity, spacing	passive block + leads
Inductor	inductance, current, saturation	winding/core	magnetic clearance	magnetic block + leads
Transformer	turns ratio, isolation, VA	winding, core, impregnation	creepage, thermal, mounting	larger magnetic block + lugs
Fuse	rating, package, holder	fuse/clip selection	replacement access	protection block + clips
Circuit breaker	trip curve, rating, terminals	DIN/panel package	terminal torque, access	protection block + line/load terminals

9. Student Tasks

Task A — Complete the Boolean decision

1. Explain why **C only** almost reaches readiness but fails `X`.
2. Explain why **A+C** is a better two-program pairing than **B+C** for a safety-critical engineering workflow.
3. Explain why the hybrid **A+B+C** becomes ready only after adding the Python/KiCad geometry layer.

Task B — Modify the demo

Change the relay origin to:

```
"relay": BlockComponent(... origin=(72, 24, 0), ...)
```

Then run the demo again.

Question:

```
Which component does it collide with, and what design change would resolve the
collision?
```

Expected reasoning:

```
The relay is likely to collide with the transformer envelope.
Resolve by offsetting one block, changing package size, rotating placement, or
introducing a keep-out zone.
```

Task C — Add a new design rule

Add a rule:

```
energy_storage components must be at least 5 mm away from magnetic components
```

Implement this as a function:

```
def category_clearance_rule(parts, category_a, category_b, required_clearance_mm):
    ...
```

Task D — KiCad extension challenge

Replace the simple text-label export with real footprints or footprint proxies:

1. Create rectangular silkscreen outlines for each block.
2. Add pad proxies for each interface.
3. Store `material`, `process`, and `notes` as text fields or external JSON metadata.
4. Run KiCad DRC manually after import.

---

10. Final Recommendation

```
Best generative hardware design toolchain:
    A+B+C hybrid + Python geometry kernel + KiCad export/placement layer
```

Boolean form:

```
READY = A ∧ B ∧ C ∧ PythonGeometry ∧ KiCadBridge
```

Engineering form:

```
safe generator
+ compact deterministic enumerator
+ configurable fabric
+ 3D block model
+ dimension-matching rules
+ collision/clearance rules
```

```
+ KiCad placement/export
= manual/semi-automated hardware design engineering toolchain
```

The important design insight is that the C programs supply **generative computation patterns**, while Python and KiCad supply the **hardware-engineering embodiment**: geometry, interfaces, footprints, placement, documentation, and review.

Conclusion :

> { generative computation patterns } + { mathematical methods } = { target-system } design engineering toolchain